```
  hF ←
fnv1Hash32(key)
        #3
  return hashFunctions.map(h => h(hM,
hF))                                        #4
```

#1 Method `key2Positions` takes an array of hash functions as input, together with a seed to initialize these functions and the key that will be hashed. It returns the set of bit indices that needs to be updated in the Bloom filter to read/write `key`.
#2 Apply murmur hashing to the key, with a given seed.
#3 Apply fnv1 hashing to the key.
#4 We use a functional programming notation here: we create a lambda function taking a hash function `h` as input and applying `h` to the two values generated by murmur and fnv1 hashing. Then we *map* this lambda function to every element of the `hashFunctions` array: this operation transforms an array of hash functions (taking 2 integers as arguments, and producing an integer as result) into an array of integers.

4.6.4 Generate Hash Functions

In listing 4.7 we have described how, in the `key2Positions` method, we pass an array of hash functions, and use it to transform a key into a list of indices: the positions in the filter's bits array where we store the key. Now it's time to see, in listing 4.8, how we initialize these `k` hash functions needed to map each key (already transformed into a string)